# DocVault

## Secure Document Management Platform

## DevSecOps Project Specifications

**ING-5-SSIR**

Proposed Theme Submission

Prepared for instructor review

Project team: 3–4 students (to be completed)

Proposed scope and implementation plan for instructor approval

## Document Purpose

This document proposes the scope, technical approach, security controls, milestones, and acceptance criteria for DocVault. It is based on the course DevSecOps project requirements and is intended to accompany the theme-approval email. Items described as proposed may be refined after instructor approval.

## 1. Project Overview

**Purpose:** DocVault is a web-based document management and sharing application. Authenticated users can upload, organize, download, and share documents according to explicit permissions. Administrators manage users and platform settings.

**DevSecOps objective:** The project will secure the application throughout its lifecycle: source code, build, test, container image, deployment, and runtime observation. The implementation will provide evidence that security checks can block defined failures and that identified vulnerabilities are fixed and retested.

**Scope boundary:** The application will use fictional users and documents only. Security testing will be restricted to the project repository and environments owned or authorized by the team.

## 2. Goals and Expected Outcomes

**Functional application:** Deliver a working application with a backend, database, authentication, at least two roles, and document-management workflows.

**Security lifecycle:** Integrate automated source-code analysis, secret detection, dependency scanning, container-image scanning, dynamic application security testing, and runtime logging.

**Secure deployment:** Build a hardened, non-root container and deploy the application to a local Kubernetes cluster with restrictive security settings and network segmentation.

**Evidence-based validation:** Document vulnerabilities before and after remediation, show a policy-enforced pipeline failure and recovery, and demonstrate at least one fictional attack event in telemetry.

## 3. Application Features

**Authentication:** Users sign in and sign out. Passwords must be stored using an appropriate password-hashing mechanism; plaintext passwords must not be stored.

**Roles:** Minimum roles: User and Administrator. Users can access only documents they own or that have been explicitly shared with them. Administrators can manage users and platform-level settings.

**Document management:** Upload, list, view/download, and delete permitted documents. Document metadata is stored in the database; files are stored in a controlled storage location.

**Sharing and authorization:** Document owners can grant or revoke access where implemented. Every document operation must verify the current user's permission on the server side.

**Audit events:** Record relevant security events such as failed authentication, denied document access, rejected uploads, and administrative actions without logging passwords, tokens, or document contents.

## 4. Proposed Architecture and Technology

**Application layer:** Python Flask or FastAPI backend with a simple web interface. The final framework may be selected by the team before implementation.

**Data layer:** PostgreSQL database for users, roles, document metadata, and sharing permissions. Document files will be stored separately from database records.

**Source control and CI/CD:** GitLab repository and GitLab CI pipeline. GitHub Actions or Jenkins may be used if approved by the instructor.

**Security tools:** Semgrep for SAST, Gitleaks for secret scanning, Trivy for dependency and container-image scanning, and OWASP ZAP for DAST.

**Container and orchestration:** Multistage Docker build using a minimal suitable base image; non-root runtime; local Kubernetes deployment using Minikube or kind.

**Observability:** Structured application logs. Prometheus, Grafana, and Loki may be added as an enhancement if time and resources permit.

## 5. Threat Model and Initial Vulnerabilities

**Threat model:** Create a STRIDE-based threat model or equivalent and map relevant findings to OWASP Top 10 categories. Identify assets, trust boundaries, threats, impact, mitigations, and validation evidence.

**Initial vulnerability set:** Prepare at least four controlled vulnerabilities spanning at least three categories. Candidate cases include broken object-level authorization (IDOR), unsafe file-upload validation, stored cross-site scripting in document metadata, and a deliberately exposed fictional test secret or insecure configuration.

**Evidence per finding:** For each finding, record the affected component or location, root cause, security impact, reproducible evidence in the team's own environment, remediation, and a regression test.

**Safe testing:** Any intentionally vulnerable behavior must be confined to a controlled test instance and must use fictional data and secrets.

## 6. CI/CD Pipeline and Blocking Policy

**Pipeline stages:** Recommended flow: checkout → build → unit/integration tests → SAST → secret scan → dependency scan → container build → image scan → deployment → DAST.

**Blocking policy:** Document severity thresholds and define which results fail the pipeline. At minimum, demonstrate a fictional secret detection or a policy-defined critical vulnerability causing a failed pipeline.

**Secret handling:** Do not commit real secrets. Runtime secrets must be supplied through protected CI/CD variables or an equivalent mechanism and must not be printed in logs.

**Remediation workflow:** Demonstrate the failing pipeline, fix the issue, rerun the checks, and retain evidence of the passing pipeline. If a fictional secret is exposed, document the proper rotation/replacement-first procedure before repository cleanup.

## 7. Container Security

**Image construction:** Use a multistage Dockerfile and a minimal suitable base image. Copy only required application artifacts into the runtime image.

**Runtime privileges:** Run as a non-root user, avoid unnecessary Linux capabilities, and prevent privilege escalation where supported.

**Image hygiene:** Ensure that secrets and the .git directory are not included in the image. Scan the image and remediate or justify findings according to the written policy.

**Verification:** Provide evidence that the image builds, the application works, and the runtime process is not running as root.

## 8. Kubernetes Deployment and Hardening

**Local cluster:** Deploy to Minikube or kind, with application and database components defined in version-controlled manifests or Helm charts.

**Security settings:** Use restrictive securityContext settings, non-root execution, read-only filesystem where feasible, dropped capabilities, and resource requests/limits where practical.

**Secrets:** Do not store plaintext secret values in version-controlled manifests. Use Kubernetes Secrets with appropriate local-lab handling or another approved secret-injection method; document limitations.

**Network segmentation:** Create a default-deny NetworkPolicy and explicitly allow only required communication, such as application-to-database traffic and necessary ingress.

**Verification:** Show that the application remains functional after hardening and that unneeded network paths are restricted.

## 9. Dynamic Testing and Runtime Detection

**DAST:** Run OWASP ZAP or an approved equivalent against a live, reachable test deployment owned by the team.

**Required validation:** Dynamically confirm at least two vulnerabilities before remediation, fix them, and rerun the same tests to show that they are no longer confirmed.

**Runtime event:** Generate at least one controlled fictional attack attempt, such as an unauthorized document-access request or a rejected suspicious upload, and show the corresponding event in application or monitoring logs.

**Incident response:** Present a response plan covering detect → contain → eradicate → restore → learn. Explain what evidence triggered the response and how the application would be returned to a safe state.

## 10. Deliverables

**Repository:** Application source, Dockerfile, Docker Compose configuration for local development, CI/CD pipeline, Kubernetes manifests or Helm chart, tests, README, and safe example configuration.

**Architecture:** Annotated diagram showing source repository, CI/CD stages, security checks, image registry, Kubernetes, application, database, and observability.

**Security report:** A 6–10 page PDF describing the threat model, vulnerabilities before and after remediation, evidence, fixes, test results, limitations, and references.

**Presentation and demo:** Live demonstration of the working application, container and Kubernetes deployment, pipeline failure and recovery, security checks, DAST retest, and runtime event.

**Individual contribution:** One-page contribution sheet per student, supported by commits or other evidence and including work completed and difficulties resolved.

## 11. Acceptance Criteria

**Application:** The application works and provides authentication, at least two roles, a backend, a database, and fictional sensitive data.

**Vulnerabilities:** At least four initial vulnerabilities across at least three categories are documented, and at least two are dynamically confirmed and subsequently fixed.

**Pipeline:** A documented security policy genuinely blocks at least one prepared test case; the corrected change passes.

**Container and Kubernetes:** The container runs without root privileges; Kubernetes deployment is functional and hardened; no plaintext secrets are committed; and network segmentation preserves required functionality.

**Runtime and reporting:** At least one fictional attack event is visible in telemetry, an incident-response plan is presented, and the report contains reproducible evidence and limitations.

## 12. Proposed Milestones

**Milestone 1 — Theme and design:** Confirm team and theme approval; agree on architecture, scope, and threat model.

**Milestone 2 — Application:** Implement authentication, roles, database models, document workflows, and baseline tests.

**Milestone 3 — Containerization:** Create Dockerfile and Docker Compose setup; verify non-root runtime and image hygiene.

**Milestone 4 — CI/CD:** Automate build and tests; add the first security check and a documented blocking policy.

**Milestone 5 — Security checks:** Integrate SAST, secret scanning, dependency scanning, and image scanning; prepare controlled vulnerability cases.

**Milestone 6 — Kubernetes:** Deploy to Minikube or kind; apply securityContext, secret handling, and NetworkPolicy.

**Milestone 7 — DAST and telemetry:** Confirm and fix two vulnerabilities dynamically; demonstrate a runtime event and response plan.

**Final — Evidence and presentation:** Complete report, architecture diagram, README, contribution sheets, and live demonstration.

## 13. Team and Approval

**Team:** Team of 3–4 students, subject to the course specification.

**Theme:** First choice: DocVault — Secure Document Management Platform.

**Application summary:** A role-based document management and sharing platform with controlled uploads, document permissions, automated DevSecOps security gates, hardened Kubernetes deployment, dynamic testing, and runtime security event detection.

**Approval request:** This document describes the proposed scope and implementation plan for instructor review. Final technical choices may be adjusted following approval.

## Appendix — Proposed Live Demonstration Sequence

**1.** Show the working DocVault application and the User / Administrator roles.

**2.** Show the container image and the Kubernetes deployment running.

**3.** Introduce a prepared fictional secret or policy-blocked test issue in a controlled branch.

**4.** Push the change and demonstrate the CI/CD pipeline failing with a safe, redacted finding.

**5.** Fix the issue, rerun the pipeline, and show it passing.

**6.** Run a DAST test against the team's test deployment and show a before/after result for a remediated vulnerability.

**7.** Trigger a controlled unauthorized document-access attempt and show the resulting log event.

**8.** Explain containment, remediation, restoration, and lessons learned.

**Important:** All test data and secrets must be fictional. Scans and attack simulations must target only the team's own application and authorized test environment. A passing pipeline does not guarantee that an application is completely secure.